



O T U S

Онлайн образование

otus.ru

 Проверить, идет ли запись

Меня хорошо видно && слышно?



Работа с файлами, grep sed awk...

Linux Professional

Андрей Буранов



*Системный администратор в компании VK, департамент VKPlay.
Работаю с операционной системой Linux более 10 лет.
Преподаю на курсах Linux более 8 лет.
Топ-3 лучших преподавателей образовательных порталов.*

*- преподаватель и руководитель курсов
Administrator Linux.Basic
Administrator Linux. Professional
Administrator Linux. Advanced*



Правила вебинара



Активно участвуем



Задаем вопросы в чат



Вопросы вижу в чате,
могу ответить не сразу



1.5 часа



Пишем в чат



Запись в личном
кабинете

Работа с файлами и каталогами

- **cat tac** вывести файл целиком
- **head tail** вывести начало и конец файла
- **sort uniq** сортировка и удаление повторов
- **wc** счётчик строк, слов и байт в тексте
- **grep** поиск по образцу
- **rev** перевернуть строку
- **paste** объединить файлы построчно
- **cut** вырезать данные из текста
- **tr** замена или удаление символов

Работа с файлами и каталогами

Вывод текстовой информации из файла на экран

cat [**<опции>**] **<имя_файла>**

Наиболее используемые опции команды cat:

- b** — пронумеровать все непустые строки
- n** — пронумеровать все строки
- s** — отобразить несколько подряд идущих пустых строк в виде одной пустой строки
- T** — показать символы табуляции, отобразив их как «^|»
- E** — показать символы конца строки как \$

Вывод информации в обратном порядке

tac **<имя_файла>**

Работа с файлами и каталогами

Если нужно отображать не весь файл, то применяют команды `head` и `tail`:

```
head [<опции>] <имя_файла1> [<имя_файла2> ...]
```

- n** **<число>** — вывести указанное число строк
- с** **<число>** — вывести указанное количество байт
- q** — не печатать заголовки файлов перед выводом
(при выводе нескольких файлов сразу)

```
tail [<опции>] <имя_файла>
```

- n** **<число>** — вывести указанное число строк
- с** **<число>** — вывести указанное количество байт
- f** — войти в постоянный цикл по считыванию
конца файла

Работа с файлами и каталогами

Поиск файла, который будет запущен при выполнении данной команды

which mc

Поиск файлов для заданной команды - **whereis**

whereis [<опции>] <команда>

- m — поиск man-страниц.
- b — поиск двоичных файлов.
- s — поиск исходных файлов.
- M — список каталогов для поиска man-страниц.
- B — список каталогов для поиска двоичных файлов.
- S — список каталогов для поиска исходных файлов.

Работа с файлами и каталогами

Linux pipe |

Pipe (англ.) — труба.

Используется для связи нескольких команд путем перенаправления вывода одной команды (stdout) на вход (stdin) последующей.



Работа с файлами и каталогами

Подсчёт количества строк, слов и символов в тексте:

wc [<опции>] [<имя_файла>]

- l** — подсчёт только количества строк
- w** — подсчёт только количества слов
- c** — подсчёт только количества символов
- b** — подсчёт только количества байт

Если текстовые данные передаются по конвейеру или перенаправлены в `stdin`, то отображается только счётчик.

```
$ cat metadata.csv | wc
43      405     5094
$ cat metadata.csv | wc -l
43
```

Работа с файлами и каталогами

paste

`paste` объединяет вместе два разных файла в один многоколоночный файл.

```
$ cat ingredients
eggs
milk
butter
tomatoes
$ cat prices
1$
1.99$
1.50$
2$/kg
$ paste ingredients prices
eggs      1$
milk      1.99$
butter    1.50$
tomatoes  2$/kg
```

Работа с файлами и каталогами

По умолчанию `paste` использует разделитель табуляции, но его можно изменить с помощью параметра `-d`.

```
$ paste ingredients prices -d:  
eggs:1$  
milk:1.99$  
butter:1.50$  
tomatoes:2$/kg
```

Ещё один распространённый способ использования `paste` — объединение всех строк в потоке или файле с помощью заданного разделителя, используя комбинацию аргументов `-s` и `-d`.

```
$ paste -s -d, ingredients  
eggs,milk,butter,tomatoes
```

Работа с файлами и каталогами

tr

`tr` расшифровывается как *translate*. Эта команда заменяет одни символы на другие. Она работает либо с символами, либо с *классами* символов, такими как строчные, печатные, пробелы, буквенно-цифровые и т. д.

На стандартных входных данных `tr <char1> <char2>` заменяет все вхождения `<char1>` на `<char2>`.

```
$ echo "Computers are fast" | tr a A  
computers Are fAst
```

`tr -c SET1 SET2` преобразует любой символ, *не* входящий в набор SET1, в символы набора SET2. В следующем примере все символы, кроме указанных гласных, заменяются пробелами.

```
$ echo "computers are fast" | tr -c '[aeiouy]' ' '  
о  и е   а е а
```

Работа с файлами и каталогами

`tr -d` удаляет указанные символы, а не заменяет их. Это эквивалент `tr <char> ''`.

```
$ echo "Computers Are Fast" | tr -d '[:lower:]'  
C A F
```

`tr` также может заменить диапазоны символов, например, все буквы между *a* и *e* или все числа между 1 и 8, используя нотацию `s-e`, где `s` — начальный символ, а `e` — конечный.

```
$ echo "computers are fast" | tr 'a-e' 'x'  
xomputxrs xrx fxst  
$ echo "5uch l337 5p34k" | tr '1-4' 'x'  
5uch lxx7 5pxxk
```

Работа с файлами и каталогами

cut

`cut` извлекает часть файла (или, как обычно, входного потока). Команда определяет разделитель полей (который разделяет столбцы) с помощью опции `-d`, а порядковые номера столбцов для извлечения с помощью опции `-f`.

Например, следующая команда извлекает первый столбец из последних пяти строк нашего CSV-файла.

```
$ tail -n 5 metadata.csv | cut -d , -f 1
mysql.performance.user_time
mysql.replication.seconds_behind_master
mysql.replication.slave_running
mysql.replication.slaves_connected
mysql.performance.queries
```

Поскольку мы имеем дело с CSV, то столбцы разделяются запятой, а за извлечение первого столбца отвечает опция `-f 1`.

cut -b 1,2,3 file1 – Вывести 3 символа строки

Работа с файлами и каталогами

sort [<опции>] <имя_файла>

- f** — игнорировать регистр
- g** — сортировать по цифрам
- r** — сортировка в обратном порядке
- u** — выводить уникальные строки

Удаление дубликатов в выводе - **uniq**

uniq <опции> [<вход>] [<выход>]

- c** — перед каждой строкой выводится количество её дубликатов
- d** — выводить только дубликаты
- u** — выводить только уникальные строки

Работа с файлами и каталогами

Разбиение файла на части - **split**:

split [<опции>] [<имя_файла>]

- b** — размер частей в байтах, на которые надо разбить файл
- l** — размер частей в строках, на которые надо разбить файл
- n** — количество частей, на которые надо разбить файл
- d** — использовать числовой префикс вместо символьного

Работа с файлами и каталогами

fold

Команда `fold` сворачивает все входные строки до заданной ширины. Например, может быть полезно убедиться, что текст помещается на дисплеях небольшого размера. Так, `fold -w n` укладывает строки по ширине `n` символов.

```
$ cat ~/Documents/readme | fold -w 16
Thanks again for
  reading this bo
ok!
I hope you're fo
llowing so far!
```

Работа с файлами и каталогами

Команда `fold -s` будет разбивать строки только на символах пробела. Её можно объединить с предыдущей, чтобы ограничить строки заданным количеством символом.

```
Thanks again  
for reading  
this book!  
I hope you're  
following so  
far!
```

Работа с файлами и каталогами

grep

Название представляет собой акроним английской фразы «*search globally for lines matching the regular expression, and print them*» — «искать везде строки, соответствующие регулярному выражению, и выводить их».

Команда `grep` ищет заданную строку текста там, где вы скажете: в файле или стандартном потоке.

Есть ещё `egrep`, `fgrep`, `pgrep`, `zgrep`. Раньше это были разные грепы, теперь все фичи* доступны в обычном `grep`.



Стандартные потоки Linux: `STDIN`, `STDOUT`, `STDERR`.
Ввод, вывод, ошибки.

Работа с файлами и каталогами

Настоящая шпаргалка это команда **man grep**. Здесь частоупотребимые опции.

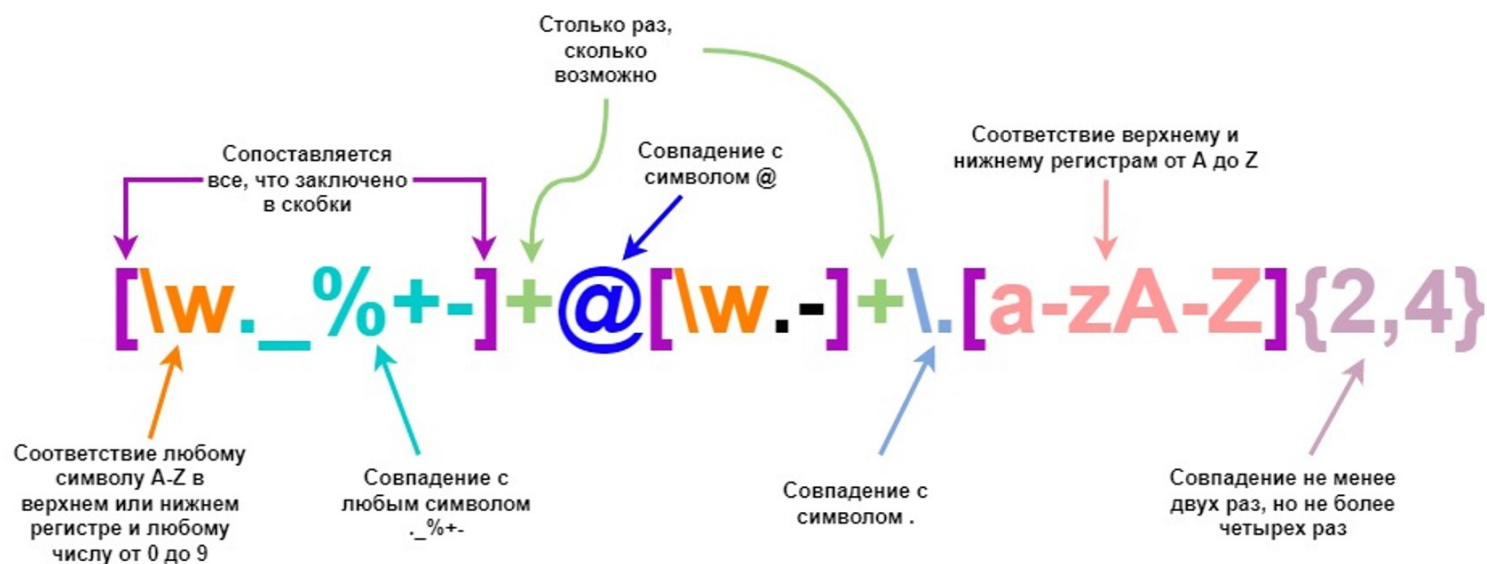
```
# Поиск строк, в которых встречается слово otus
grep 'otus' file
# Поиск строк, в которых встречается слово otus без учёта регистра
grep -i 'otus' file
# Подсчёт количества строк, в которых встречается ключевое слово otus
grep -c 'otus' file
# Поиск номера строки, в которой находится ключевое слово otus
grep -n 'otus' file
# Поиск строк, в которых строка otus отсутствует
grep -v 'otus' file
# Найти строки со словом otus, и вывести ещё 2 после каждой найденной строки A – after
grep -A2 'otus' file
# Найти строки со словом otus, и вывести ещё 2 перед каждой найденной строкой B – before
grep -B2 'otus' file
# Найти строки со словом otus, и вывести 2 строки до и 2 после найденных строк C – context]
grep -C2 'otus' file
# рекурсивный обход всех файлов в каталоге для поиска строк, в которых встречается слово otus
grep -r 'otus'
```

Работа с файлами и каталогами

Шпаргалка по регуляркам

[0-9]	— Любая цифра
[a-z][A-Z][a-zA-Z]	— 3 примера сразу: строчные буквы, прописные буквы, и те и другие буквы
^[^0-9][^a-zA-Z]	— 2 примера: всё кроме цифр, всё кроме букв (“крышечка” – отрицание)
{2-5}	— Модификатор “число повторений”
[0-9]{2}	— Число из двух цифр
[a-яA-Я]{2,5}	— Слово не короче двух и не длиннее пяти букв
^	— Начало строки
\$	— Конец строки
^Otus	— Строка начинается со слова Otus
Otus\$	— Строка заканчивается словом Otus
^Otus\$	— Строка состоит из слова Otus и больше в ней ничего нет
.	— заменяет любой символ (любой <u>один</u> символ)
*	— модификатор “ноль или больше”
+	— модификатор “один или больше”
O.us	— Подходит: Otus, Opus, Oous. Не подходит: Ous, Ottus
O.*us	— Подходит: Otus, Opus, Oous, Ottus, Ous. Не подходит: OttUS Otsu
O.+us	— Подходит: Otus, Opus, Oous, Ottus. Не подходит: Ous Otsu
[0-9]*	— Ноль или больше цифр
[0-9]+[a-zA-Z]{2}	— Как минимум 1 цифра (или больше одной) и ровно 2 буквы
(42 31)	— Вертикальная черта означает “или”. В данном примере ищем или 42 или 31

Работа с файлами и каталогами



Регулярное выражение для поиска e-mail адреса

Работа с файлами и каталогами

AWK. Поточный текстовый редактор

AWK — си-подобный сценарный язык построчного разбора и обработки входного потока (например, текстового файла) по заданным шаблонам (регулярным выражениям). Может использоваться в сценариях командной строки.

Название AWK складывается из первых букв фамилий разработчиков языка — Ахо, Уайнбергера (англ. *Peter J. Weinberger*) и Кернигана. Первая версия была написана в 1977 году в AT&T Bell Laboratories.

Работа с файлами и каталогами

`awk` — это чуть больше, чем просто инструмент обработки текста: на самом деле у него **целый язык программирования**. В чём `awk` действительно хорош — так это в разбиении файлов на столбцы, и делает это с особенным блеском, когда в файлах перемешаны пробелы и табы.

```
$ cat -t multi-columns
John Smith      Doctor^ITardis
Sarah-James Smith^I    Companion^ILondon
Rose Tyler      Companion^ILondon
```

Примечание. `cat -t` отображает табы как `^I`.

Как видим, столбцы разделены либо пробелами, либо табуляциями, и не всегда одинаковым количеством пробелов. `cut` здесь бесполезен, потому что работает только с одним символом-разделителем. Но `awk` легко разберётся с таким файлом.

Работа с файлами и каталогами

Шпаргалка по AWK

`awk '{print $1}'`

– напечатать первый блок

`awk '{print $1 $2}'`

– напечатать первый и второй блоки

`awk '{print $2,$NF}'`

– напечатать первый и последний блоки

`awk 'FS=":" {print $2,$NF}'`

– аналог если разделитель двоеточие

`awk 'FNR==2 {print $0}'`

– напечатать вторую строку

`awk '{print $0}' ORS=' '`

– напечатать файл в одну строку

Переменные:

FS – Разделитель полей (по умолчанию пробел)

NF – Количество строк

FNR – Количество строк в файле

ORS – Выходной разделитель

Работа с файлами и каталогами

SED. Поточный текстовый редактор

Sed (от **англ. Stream *ED*itor**) — потоковый текстовый редактор (а также язык программирования), применяющий различные predetermined текстовые преобразования к последовательному потоку текстовых данных.



Шпаргалка по sed

`sed -n 10p` – показать десятую строку

Команда `d` – удалить

`sed -e '10 d'` – удалить десятую строку

`sed -e '2,4 d'` – удалить 2,3,4 строки

`sed -e '/pts/ d'` – удалить строки в которых есть **pts**

`sed -e '/^$/d;/^s*#/d'` – удалить пустые и закомментированные строки

Команда `s` – заменить

`sed -e "s/USER/user/g"` – заменить `USER` на `user`

`sed -e "1,3 s/USER/user/"` – заменить `USER` на `user` только в строках 1,2,3

`sed -e "3,$ s/USER/user/"` – заменить начиная с третьей строки и до конца

Изменение файла

`sed -i.bak '<command>' file.txt` – выполнить команду `<command>` и заменить содержимое файла `file.txt`

`sed -i.bak '<command>' file.txt` – заменить содержимое файла и сохранить бекап в `file.txt.bak`

Работа с файлами и каталогами

Замена части строки в файле:

Есть файл test.txt вида
много текста
PrivateKey = <insert_key>
много текста

sed -i.bak '/^PrivateKey =/s/<insert_key>/123456/' test.txt

много текста
PrivateKey = 123456
много текста

Работа с файлами и каталогами

Поиск файлов по файловой системе командой **find**

`find [папка] [параметры] критерий шаблон [действие]`

Основные параметры:

- maxdepth — максимальная глубина поиска по подкаталогам.
- depth — искать сначала в текущем каталоге, а потом в подкаталогах.
- mount — искать файлы только в этой файловой системе.
- print — выводить полные имена файлов
- type f — искать только файлы
- type d — искать только каталоги

Основные критерии:

- name - поиск файлов по имени
- perm - поиск файлов в Linux по режиму доступа
- user - поиск файлов по владельцу
- mtime - поиск по времени модификации файла
- newer - найти файлы новее чем указанный
- size - поиск файлов в Linux по их размеру

Работа с файлами и каталогами

Удаление файлов по маске игнорируя регистр символов

```
find . -type f -name -iname "*.LnK*" -delete
```

Удаление пустых папок

```
find . -type d -empty -delete
```

```
find . -type d -empty -exec rmdir {} \;
```

Шпаргалка по команде find

<code>find . -type f</code>	— найти все файлы
<code>find . -type d</code>	— найти все директории
<code>find . -maxdepth 2 -mindepth 1</code>	— ограничить глубину поиска
<code>find . -type f -mtime +10</code>	— найти файлы с mtime старше 10 дней
<code>find . -type f -mtime +10 -mtime -20</code>	— найти файлы с mtime в диапазоне от 10 до 20 дней
<code>find . -type f -delete</code>	— удалить найденное
<code>find . -type f -exec echo "founded {}" \;</code>	— передать найденное внешней команде
<code>find ./* -type f xargs -0 -i mv {} {}.rename</code>	— обработать найденное через xargs

Работа с файлами и каталогами

| xargs

Принцип работы xargs можно описать следующим образом: программа берет данные из стандартного ввода или из файла, разбивает их в соответствии с указанными параметрами, а затем передает другой программе в качестве аргумента.

В общем виде синтаксис команды xargs можно представить так:

```
[команда_генератор_списка] | xargs [опции_xargs] [команда]
```

Одна из самых частых ситуаций, в которых используется xargs — удаление файлов, найденных при помощи команды find.

Работа с файлами и каталогами

```
$ ls
one.sh one.py two.sh two.py

$ find . -name "*.sh"| xargs rm -rf

$ ls
one.py two.py
```

Отметим, что операцию удаления файлов можно осуществить и без `xargs`, а с помощью команды

```
$ find . -name "*.sh" -exec rm -rf '{}' \
```

Описанный способ не работает, если в имени одного из удаляемых файлов содержится пробел. Имя, состоящее из двух слов, разделенных пробелом, не будет воспринято как единое целое.

Работа с файлами и каталогами

```
$ ls
new file.sh one.sh one.py two.sh two.py

$ find . -name "*.sh" | xargs rm -rf

$ ls
new file.sh one.py two.py
```

Как видим, файл, в имени которого имеется пробел, не был удалён.

Чтобы решить эту проблему, используется опция `print0` для команды `find` и опция `-0` для команды `xargs`. Она заменяет стандартный разделитель (перенос строки на нуль-символ (`\x0`), который и означает конец хранимой строки:

```
$ find . -name "*.sh" -print0 | xargs -0 rm -rf
```

Работа с файлами и каталогами

Все эти инструменты открывают целый мир возможностей, так как позволяют извлекать и преобразовывать данные, создавая целые конвейеры из команд, которые, возможно, никогда не предназначались для совместной работы.

Каждая из них выполняет относительно небольшую функцию (сортировка `sort`, объединение `cat`, фильтры `grep`, редактирование `sed`, вырезание `cut` и т. д.).

Вопросы для проверки

Вопросы для проверки

Какой поток в Linux обозначается цифрой 2?

- A. stdin
- B. /dev/null
- C. stdout
- D. stderr



Отправьте в чат верную букву

Вопросы для проверки

Какой поток в Linux обозначается цифрой 2?

- A. stdin
- B. /dev/null
- C. stdout
- D. stderr**

Вопросы для проверки

Что делает команда `awk '{print $1}' access.log`?

- A. Считает количество строк
- B. Выводит первый столбец
- C. Фильтрует строки с первым полем
- D. Удаляет дубликаты



Отправьте в чат верную букву

Вопросы для проверки

Что делает команда `awk '{print $1}' access.log`?

A. Считает количество строк

B. Выводит первый столбец

C. Фильтрует строки с первым полем

D. Удаляет дубликаты

Вопросы для проверки

Какой флаг grep позволяет показать строки после совпадения?

- A. -A
- B. -B
- C. -C
- D. -n



Отправьте в чат верную букву

Вопросы для проверки

Какой флаг grep позволяет показать строки после совпадения?

A. -A

B. -B

C. -C

D. -n

Вопросы для проверки

Что делает команда `command 2>&1 | grep "fail"`?

- A. Ищет только стандартный вывод
- B. Фильтрует только stderr через grep
- C. Объединяет stdout и stderr и фильтрует всё
- D. Отправляет ошибки в /dev/null



Отправьте в чат верную букву

Вопросы для проверки

Что делает команда `command 2>&1 | grep "fail"`?

- A. Ищет только стандартный вывод
- B. Фильтрует только stderr через grep
- C. Объединяет stdout и stderr и фильтрует всё**
- D. Отправляет ошибки в /dev/null

Вопросы для проверки

Какой оператор Bash перезаписывает файл при выводе?

- A. >>
- B. >
- C. |
- D. <



Отправьте в чат верную букву

Вопросы для проверки

Какой оператор Bash перезаписывает файл при выводе?

A. >>

B. >

C. |

D. <

Вопросы?



Ставим "-",
если вопросы есть



Ставим "+",
если вопросов нет



Подведение итогов



Заполните, пожалуйста, опрос о занятии

Спасибо за участие!

